bit; then these `k` bits are all set to `1`. Notice how the two triplets of indices generated by x_1 and x_2 partially overlap (they both point to the sixth element. For a more detailed example of how insertion works, see figure 4.5.
(C) Likewise, to check if an element y_i is in the set, it's hashed `k` times, obtaining just as many indices; then we read the corresponding bits, and if and only if all of them are set to `1`, we return `true`. In the bottom drawing, element y_1 appears to be in the set (but we can't exclude that the filter is returning a false positive), while element y_2 can't be in the set, because one of the indices generated from its hashing holds a `0`. Check figure 4.4 for further insight on how lookup works.

Ideally, we would need k different independent hash functions, so that no two indices are duplicated for the same value. It is not easy to design a large number of independent hash functions, but we can get good approximations. There are a few solutions commonly used:

- Use a parametric hash function `H(i)`: this meta-function, which is a generator of hash functions, takes in input an initial value `i` and outputs a hash function H_i=`H(i)`; during bloom filter's initialization we can create `k` of this functions, H_0 to H_{k-1}, by calling the generator `H` on `k` different (and usually random) values.
- Use a single hash `H` function, but initialize a list `L` of `k` random (and unique) values; for each entry `key` that is inserted/searched, create `k` values by adding or